

EECE7352 Computer Architecture

Homework 3

Sakthivel P. Sivakumar

NUID: 002312294

Part D: (20 Points)

As part of your reading, we have provided a copy of the original paper by Robert Tomasulo that describes the Tomasulo Algorithm, as implemented on the IBM 360 Model 91. We also have a link to a YouTube video that describes how the algorithm works. You are asked to read the article and review the video and then answer the following questions.

1. Discuss the role of the Common Data Bus (CDB) in the Tomasulo Algorithm. How is this mechanism increasing instruction throughput?

Ans: The Common Data Bus is the central figure in the Tomasulo Algorithm that enables out-of-order execution while maintaining correct data dependencies. Its main role is to act as a broadcast bus that connects the outputs of all execution units like the adder and multiplier/divider and data buffers directly to the inputs of all functional units and the floating-point registers.

Working of CDB:

- 1) Resolving Dependencies with Tags:** When an instruction is issued, it's sent to a free reservation station. If an operand is not yet available, the reservation station doesn't get the value. Instead, it gets a tag identifying which execution unit or buffer will produce that value. The reservation station "listens" to the CDB, waiting for that specific tag to be broadcast.
- 2) Broadcasting Results:** When an execution unit (e.g. adder) finishes its calculation, it requests access to the CDB. Once granted, it broadcasts both its result and its unique tag to all reservation stations and registers simultaneously.
- 3) Increasing Throughput:** The CDB, combined with reservation stations, allows the processor to hide the latency of slow operations. While a long-latency instruction is busy executing, other independent instructions can be issued, grab their operands, execute, and write their own results to the CDB.

By enabling Out-of-Order Execution, Instructions do not have to wait in a queue. They begin executing as soon as their operands are available. This breaks the "in-order" execution bottleneck, significantly increasing instruction throughput.

2. Assume that we want to integrate the Tomasulo Algorithm in our 5-stage RISC-V pipeline. Provide a code snippet (at least 6 instructions) of RISC-V code that includes multiple data dependencies. Show how your design with an integrated Tomasulo Algorithm can hide the latencies that would normally be encountered in the RISC V pipeline.

Ans:

To write a RISC-V code with multiple data dependencies, I have taken the example of calculating $(A*B) + (C*D) - E$, assuming $x1=A$, $x2=B$, $x3=C$, $x4=D$, $x5=E$

Code Snippet:

```
MUL x6, x1, x2    # x6 = A * B
MUL x7, x3, x4    # x7 = C * D
ADD x8, x6, x7    # x8 = (A*B) + (C*D)
SUB x9, x8, x5    # x9 = x8 - E
ADDI x10, x9, 100 # x10 = x9 + 100
SW x10, 0(x11)    # Store result
```

Read After Write (RAW): Instruction 3,4,5,6 depend on previous results

Assumptions for timing:

The Baseline core: classic 5-stage (IF, ID, EX, MEM, WB), in-order, single EX stage, MUL is a multi-cycle EX op that occupies EX for 5 cycles, these are non – pipelined in the baseline.

ADD/SUB/ADDI all these take 1 EX cycle, SW does 1 EX cycle (address) + 1 MEM cycle. With Tomasulo integrated, We have two integer multiplier functional units (each 5-cycle latency, pipelined), One integer ALU (1-cycle), One LSU, Reservation Stations (RS), a Reorder Buffer (ROB), and a single CDB (one result can broadcast per cycle). In-order commit from the ROB (precise state), but out-of-order execute.

Without Tomasulo:

In the baseline pipeline the first MUL ties up EX for 5 cycles; nothing else that needs EX can proceed, even if there are no true dependencies. The sequence is effectively serialized through EX:

EX usage: MUL (5) + MUL (5) + ADD (1) + SUB (1) + ADDI (1) + SW addr (1) = **14 EX cycles**,

along with these the ID/MEM and WB overheads make the total no of cycles to 18.

With Tomasulo:

With the help of reservation stations + register renaming, we can issue all the 6 instructions. The two MULs grab different multiplier units and run in parallel. Downstream operations wake up as soon as both of their operands arrive, regardless of original order.

To achieve this, the pipeline is being upgraded with several key components. First, it's getting a set of Reservation Stations (RS), which are split into three stations for ADD/SUB instructions, two stations for MUL instructions, and a two-entry buffer for handling loads and stores.

A Common Data Bus (CDB) is also being added, which is constantly broadcasting finished results to all of these stations at once.

The design is also implementing Register Renaming. This system is using tags (like V1, V2, etc.) to track future values, so an instruction doesn't have to stall; it just waits for the tag to appear on the CDB.

Finally, a Reorder Buffer (ROB) is being included. Its crucial job is ensuring that even though instructions are executing out-of-order, they are committing their results in the correct, original program order.

By executing the independent instructions and forwarding results as soon as they are ready, the out-of-order core compresses the entire sequence down to a **Total Cycles to 11**.

Latency Hiding Techniques:

- **Instruction-Level Parallelism (ILP):** The processor exploits ILP by executing instructions 1 and 2 simultaneously, as there are no false dependencies between them.
- **Non-blocking Issue:** The pipeline does not stall while waiting for operands, which is demonstrated by all 6 instructions being successfully issued by cycle 6.
- **Dynamic Scheduling:** Instructions (like instruction 3) wait for both of their operands independently and only begin execution as soon as both are ready, not before.
- **Register Renaming:** This technique removes false dependencies by eliminating both Write-After-Write (WAW) and Write-After-Read (WAR) hazards, leaving only true Read-After-Write (RAW) data dependencies.

In conclusion, the Tomasulo algorithm eliminated 12 stall cycles, cutting execution time from 18 down to 11 cycles. This **64% speedup** on the example code highlights the key insight: **Tomasulo converts pipeline stalls into reservation station waits**. This change allows independent instructions to proceed, fully exploiting instruction-level parallelism. For more parallel workloads, this speedup can increase further, potentially up to 3-4x.

3. Besides the implementation described in the Tomasulo paper, find a more recent CPU that has implemented the Tomasulo algorithm. Discuss how the design works, and how it differs from Tomasulo's original idea.

Ans:

The Intel P6 microarchitecture, used in CPUs like the **Pentium Pro**, is a famous implementation of Tomasulo's algorithm. It works by breaking instructions into smaller **micro-operations**. It renames architectural registers, such as EAX, to a larger set of internal physical registers to eliminate false dependencies. **It then uses reservation stations and a common data bus for out-of-order execution**, just as Tomasulo proposed.

The P6, however, introduces crucial modifications. The most important is the **Re-Order Buffer (ROB)**, which holds completed results and commits them in the original program order. This ensures "precise exceptions," allowing the processor to correctly recover from an error, which was a problem in the original design. The ROB also enables **speculative execution**, where the CPU guesses branch outcomes and can easily discard the results if the guess is wrong. Finally, P6's full register renaming system is a more extensive version of Tomasulo's original tagging concept, allowing for more parallelism.